[CS3704] Intermediate Software Design and Engineering

Dr. Chris Brown
Virginia Tech
11/3/2026

Announcements

- PM2.2 due Friday (11/6) at 11:59pm
  - Progress Report
  - Black Box Test Plan
- Attending Funeral on Thursday
  - Office Hours tomorrow cancelled
  - Guest Lecture: Michael Irwin (Docker)
- Next Week
  - 11/10: Testing and Debugging Workshop
  - 11/12: Guest Lecture [Bilal Aziz, insightsoftware]
- Ut Prosim opportunities

CI/CD and Deployment

DevOps
CI/CD
Advanced Testing
Deployment Strategies
Case Study

Learning Outcomes

By the end of the course, students should be able to:

- **Understand software engineering processes, methods, and tools used in the software development life cycle (SDLC)**
- Use techniques and processes to create and analyze requirements for an application
- Use techniques and processes to design a software system
- **Identify processes, methods, and tools related to phases of the SDLC**
- Explain the differences between software engineering processes
- Discuss research questions and current topics related to software engineering
- Create and communicate about the requirements and design of a software application

Warm-Up

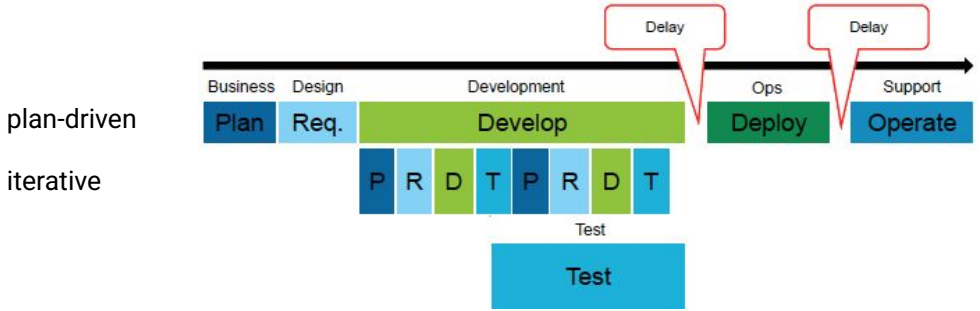
TODO: Do you have experience developing software that was delivered for other users? How did you deploy the application? How did it go?

Deployment

- The process of delivering software to users
  - *complete* implementation or *partially completed* increment.
- Historically, ***developers*** wrote the code, ***testers*** validated the program, and a separate ***operations team*** was in charge of deploying and monitoring the system.

Problems

- Teams couldn’t satisfy project goals
- Delivering a service from inception to its users is too slow and error-prone.
- Internal friction points made this the case.



➔ This loses money (delay = loss) 💸

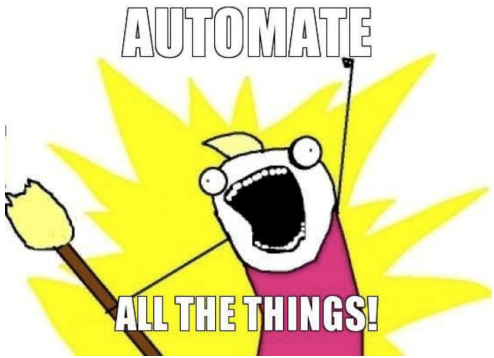
➔ IT was frequently considered the bottleneck

DevOps

Set of practices that combines software development (Dev) and IT operations (Ops) team responsibilities.

DevOps Values:

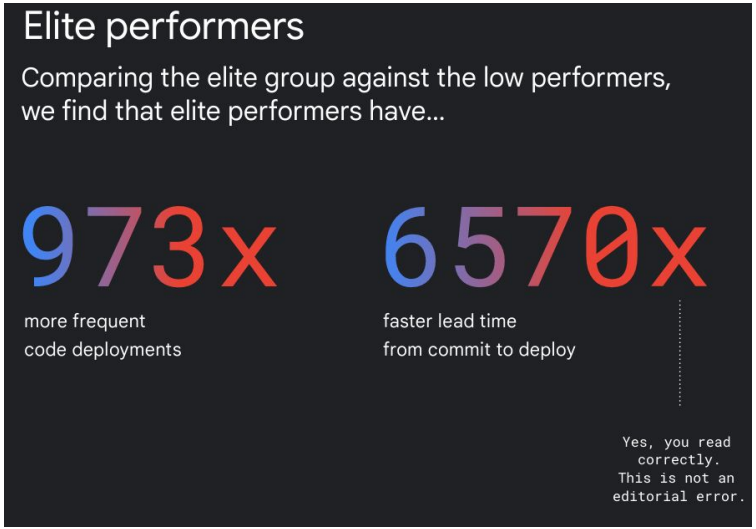
- Shared responsibilities: No team silos.
- “Testing in production”.
- Blameless culture; You are the support person.
- Move fast. Feature experimentation.
- Automate as much as possible



Impact of DevOps

Elite: Multiple deployments per day; < 1 hour lead time
High: Weekly to Monthly deployments; day-week lead time
Medium: Monthly to 6-month deployments; month-six month
Low: Longer than six months; more than six month lead time

- *Lead time:* Time from code committed to running in production

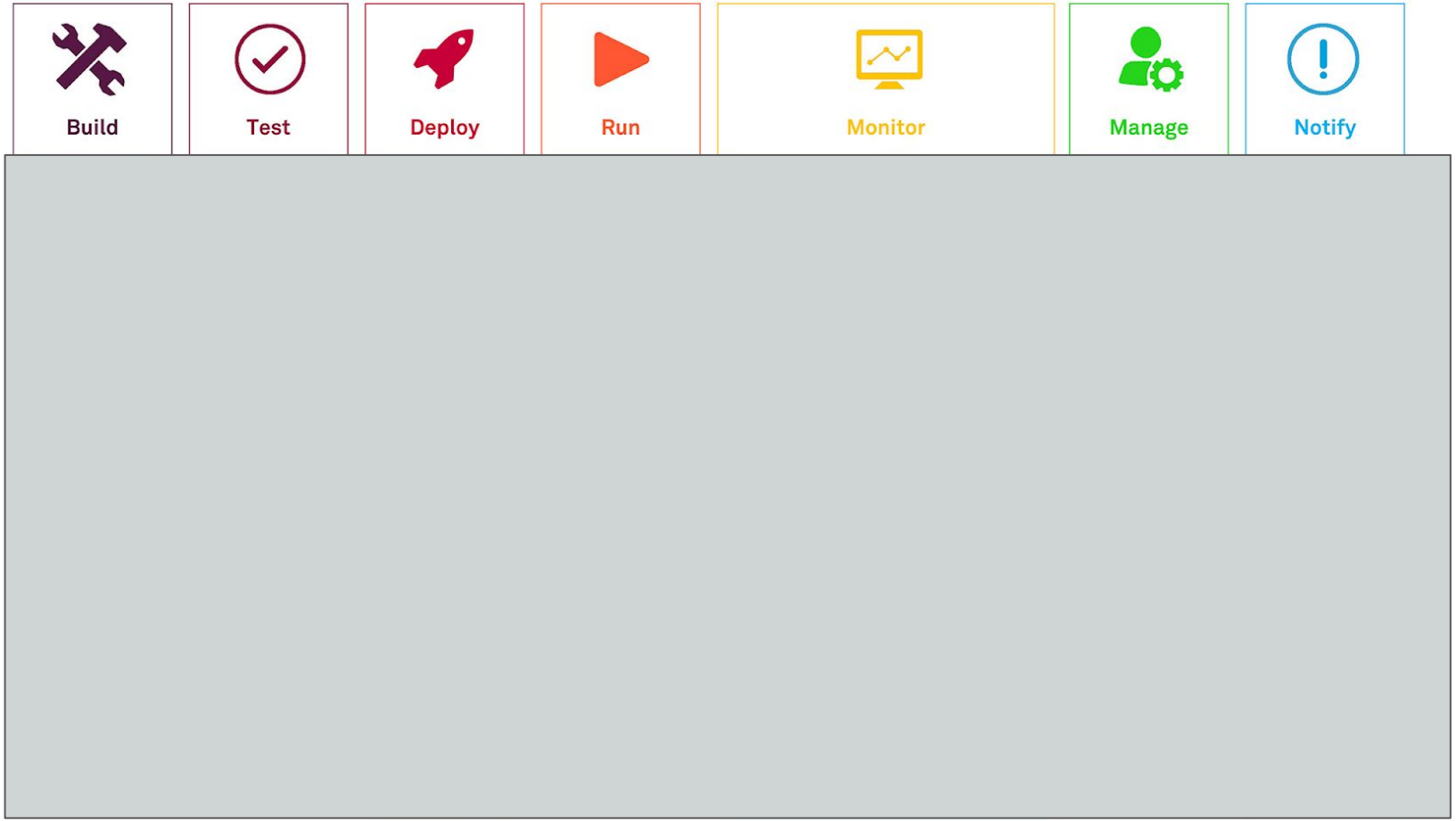


In the past, more frequent deployments meant lower quality software. Employing these practices now allows for more deployments with ***higher*** reliability and stability of applications.

Spring 2027 Class! (*CS6704: DevOps*)

Key Terms

- **Pipeline:** a series of steps that must be performed in order to deliver a new version of software.
- **Toolchain:** a set of multiple tools to accomplish varying tasks (as opposed to just one).


Build
Test
Deploy
Run
Monitor
Manage
Notify

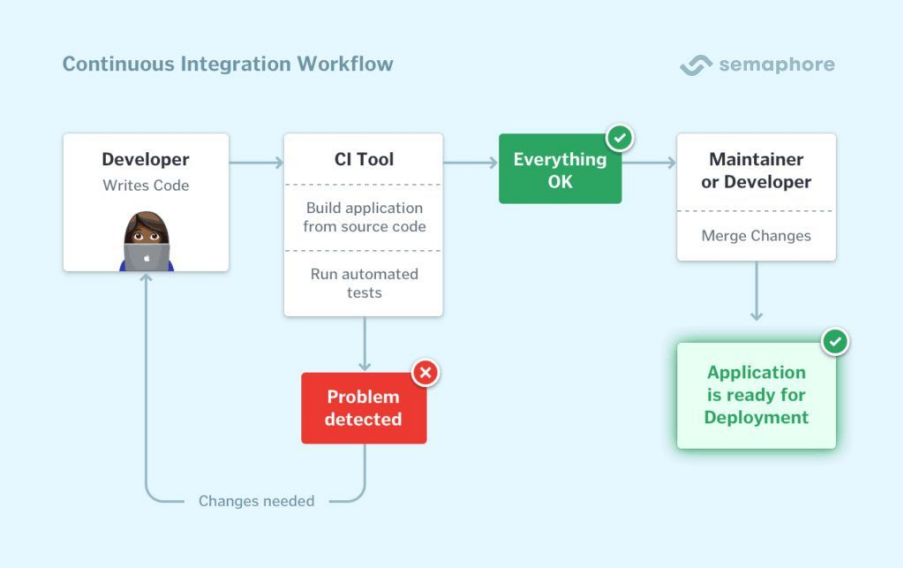
Another Problem…

Manual Integration (the process of merging changes into the source code repository) is expensive

- Integrating, testing, building, and staging can take hours or days…
- Integration problems and bugs are detected too late.
- Involves manually locking down part(s) of the production system while performing deployments

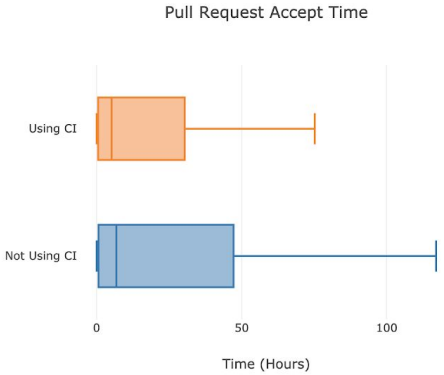
Continuous Integration

A practice where developers automatically integrate, test, build, and stage software in response to *every software change* committed to the source repository.



Continuous Integration Benefits

Reason	total (percent)
CI helps us catch errors earlier	375 (75%)
CI makes us less worried about breaking our builds	359 (72%)
CI provides a common build environment	349 (70%)
CI helps us deploy more often	339 (68%)
CI allows faster iterations	284 (57%)
CI makes integration easier	283 (57%)
CI can enforce a specific workflow	198 (40%)
CI allows testing across multiple platforms	143 (29%)
CI lets us spend less time debugging	121 (24%)
CI allows running tests on more powerful hardware	109 (22%)



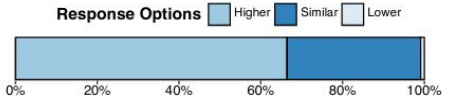


Fig. 2. Do projects with CI have (higher/similar/lower) test quality? N=444

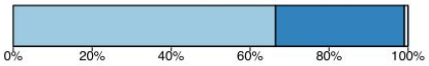


Fig. 3. Do projects with CI have (higher/similar/lower) code quality? N=438

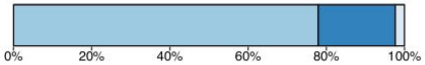


Fig. 4. Are developers on projects with CI (more/similar/less) productive? N=462

[Hilton]

Continuous Delivery vs. Deployment

- ***Continuous delivery*** focuses on automation steps and strategies to safely prepare changes for production.
  - i.e. releases, versioning, etc.
  - Keeps the code in a deployable state at any given time
  - Testing in “production-like” environment
- ***Continuous deployment*** focuses on delivery and the actual deployment across environments to distribute the application.
  - “Distribution of bits to users”

Which one is CD in CI/CD? It depends…

Another Problem…

- Too many tests are a liability
  - More difficult to update, organize, and maintain
  - Slow tests slow everyone and everything down



Especially *integration tests!*

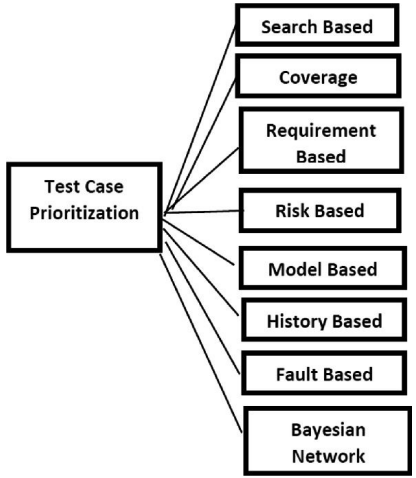
- *Reminder:* Testing groups of subsystems, and eventually the entire system, to uncover errors associated with interfacing between units.

Advanced Testing Methods

- Test Case Prioritization
- Fuzz Testing
- Mutation Testing [HW4]
- Chaos Engineering

Test Case Prioritization

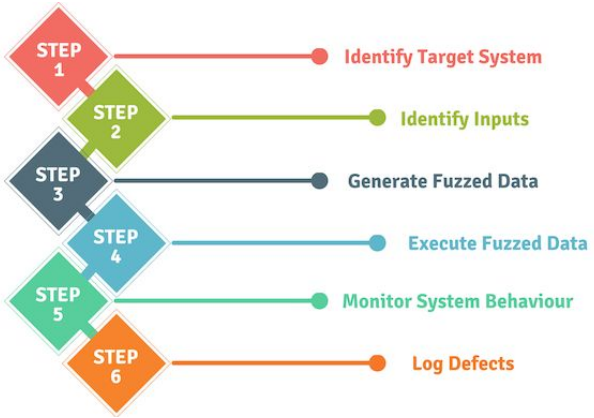
- Select specific test cases to run in a test suite on the basis of different factors
  - During a build, only run important test cases
  - Can prioritize based on code coverage, features, risk, functionality, etc.



[Yadav]

Fuzz Testing

- **Fuzzing:** The process of injecting invalid, malformed, or unexpected inputs into a system to reveal defects and vulnerabilities.
  - Heavily used for security
  - *Runtime injection*



Mutation Testing

Mutation testing is a form of whitebox testing that involves modifying the program in small ways with *mutants*, causing the behavior of the original program to change. Unit test cases should be able to detect program behavior and fail, killing the mutants.

Example mutant operators:

- Statement deletion (removing line(s) of code)
- Statement duplication or insertion (adding line(s) of code)
- Replacement of boolean expressions (switching True and False values)
- Replacement of arithmetic operators (i.e. +, -, *, /, %)
- Replacement of boolean operators (i.e. >, <, >=, <=, ==, !=)
- Replacement of variables within the same scope
- and more!

Original Code	Modified Code

if(p >q)
r= 5;
else
r= 15;
	if(p < q)
	r = 5;
	else
	r = 15;

Ch

- T nd
  u
  - C
  - C to



*Netflix did not go down during AWS outages because of its purpose-**built resilient architecture** and operational practices that minimize dependency on any single AWS region or service.*

Deployment Strategies

How is modern software actually deployed?

- Basic deployment
- Rolling deployment
- Blue-green deployment
- Canary deployment
- A/B Testing
- Shadow Deployment/Dark Launch

[Harness]

Basic Deployment

- Target environment is updated at the same time.

Pros:

- Simple, fast, and cheap
- Should only be used if your product is not critical or deploying to a lower environment

Cons:

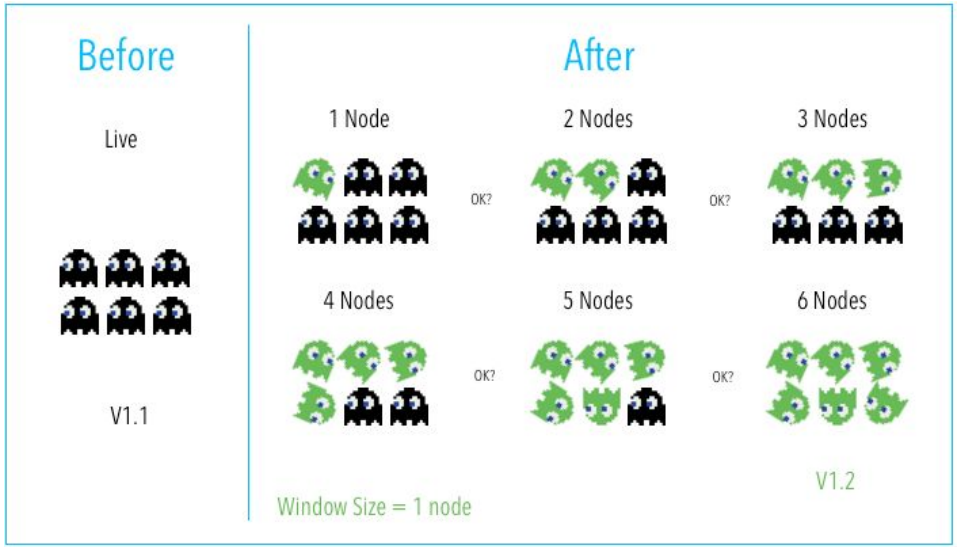
- Riskiest deployment strategy
- Difficult for rollbacks
- Not outage-proof



Rolling Deployment

- Target environments are incrementally updated in *N* batches.

+ Simpler roll back, less risk
- Requires support for old and new versions
- Slow



Blue-Green Deployment

- Utilize two identical environments, i.e. *blue* for staging and *green* for production. Testing and validation testing can done in blue environment while user traffic is shifted to green. Then switch after successful testing and deployment.

+ Simple, easy to implement, less risk
+ Straightforward roll backs (just flip to other environment)
- More cost to replicate production environment
- Shifts all user traffic at once

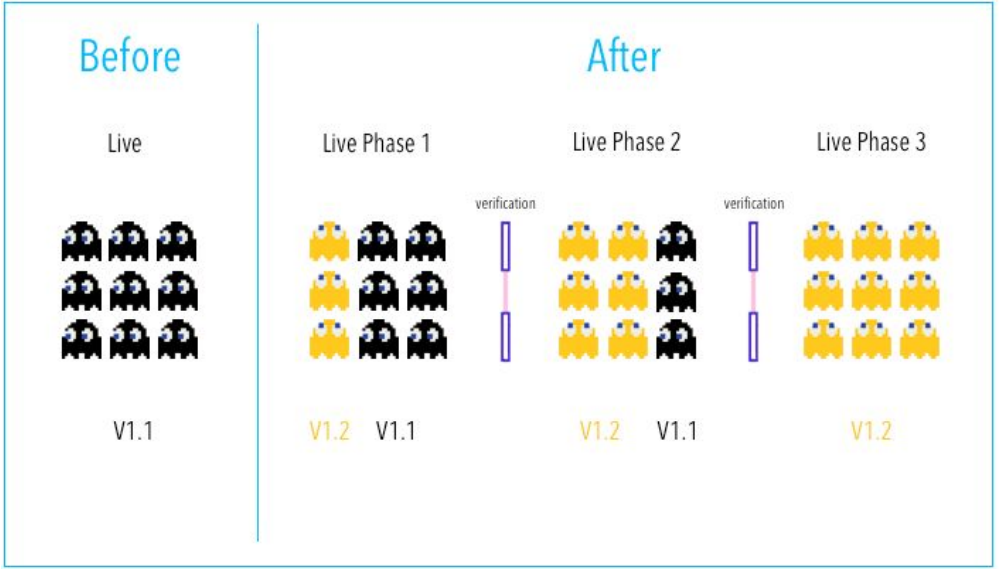


Canary Deployment

- Release an application incrementally to a subset of users in small phases.

\+ Lowest risk!
\+ Fast and safe roll backs
\+ Cheaper than blue-green
\- Testing in production needed

Note: Canary deployments got their name from an old British mining practice. Back in the day, miners used canaries to test coal mines' safety before they went in. If the canaries returned unharmed, the miners felt safe to enter. However, if something did happen to the birds, they knew that the mines were filled with toxic gases.

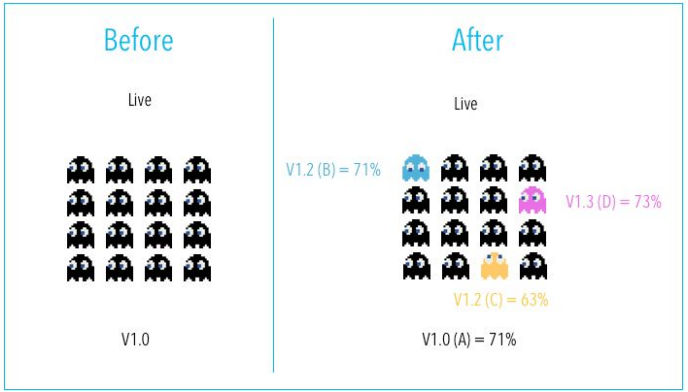


A/B Testing

- Run different versions of the same application simultaneously in the same environment for a period of time.
  - While other deployment strategies have an immediate goal of updating all nodes with a specific version, A/B testing is more focused on experimentation, exploration, and testing ideas.

\+ Easy and cheap to test new features
\- Experimental nature
\- Can be complex to automate

A/B Testing (cont.)



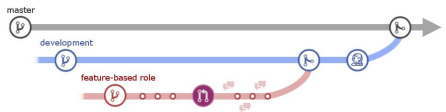
How do you run different versions of a program simultaneously?
Route user traffic to specific versions based on a given criteria to perform measurements or observe using:

- Feature flags/toggles




```
function reticulateSplines(){
  if( featureIsEnabled("use-new-SR-algorithm") ){
    return enhancedSplineReticulation();
  }else{
    return oldFashionedSplineReticulation();
  }
}
```

- Branches



- Other automated tools

Dark Launching

- The feature is available on the application, but not actually shown in the user interface.
  - Purpose is to monitor the load of the system with the new feature during day-to-day interactions.

\+ Simple to implement
\+ Tests load on system without production impact
\- No user interaction

Example at Facebook:

“The secret for going from zero to seventy million users overnight is to avoid doing it all in one fell swoop. We chose to simulate the impact of many real users hitting many machines by means of a "dark launch" period in which Facebook pages would make connections to the chat servers, query for presence information and simulate message sends without a single UI element drawn on the page. With the "dark launch" bugs fixed, we hope that you enjoy Facebook Chat now that the UI lights have been turned on.” [Obasanjo]

Which Strategy to Use?

Considerations:



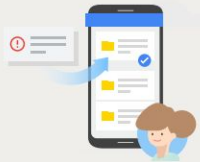
Change lead time:
the time it takes for a code commit or change to be successfully deployed to production.



Deployment frequency:
how often application changes are deployed to production.



Change fail rate:
the percentage of deployments that cause failures in production,[1] requiring hotfixes or rollbacks.



Failed deployment recovery time:
the time it takes to recover from a failed deployment.

[Accelerate]

AI for CI/CD

Can LLMs Write CI? A Study on Automatic Generation of GitHub Actions Configurations [Ghaleb]

Short Answer: **No**

Their results reveal "LLM limitations for CI configuration generation. Analyzing GPT-4o outputs reveals issues like missing or renamed steps, misinterpreted descriptions, and unnecessary additions that may affect structural and contextual correctness, indicating a gap between generation quality and the precision required for executable CI configurations."

AI for Delivery/Deployment

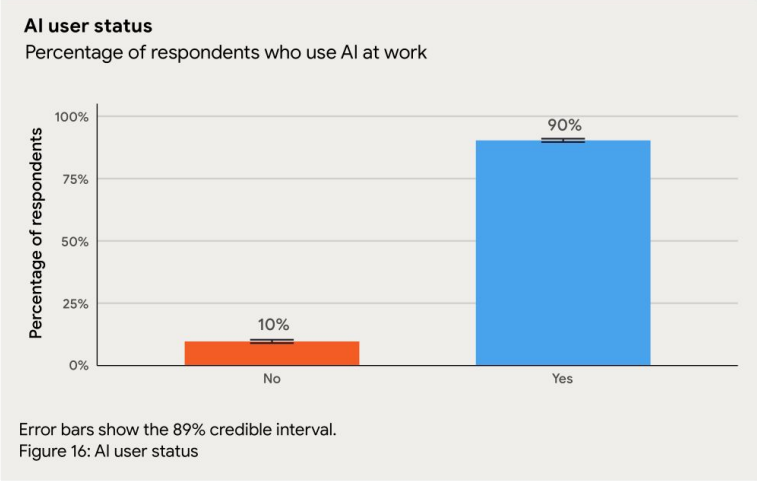


Figure 16: AI user status

Higher levels of software delivery throughput, but…

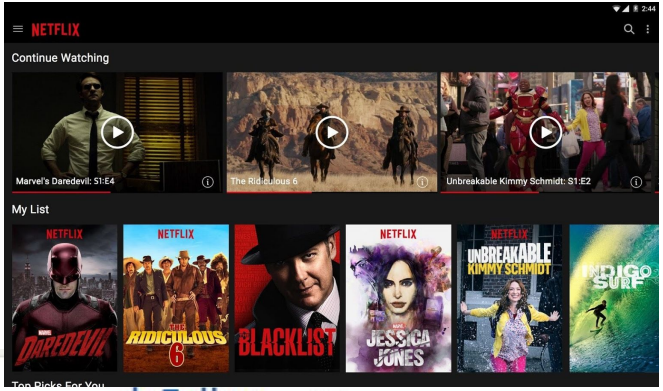
- Higher levels of software delivery instability [Accelerate]

"If perception of code quality is increasing with AI assistance but stability is dropping, it indicates a mismatch between how well we think it performs a task and how well it actually does. Given the intense focus on increasing the speed of coding, we're likely seeing suboptimization on a massive scale." [Fenton]

Case Study: Netflix

How many commits are deployed daily, at Netflix?

4,000+



Muhammad Mudassir • 2nd
Software Engineer
2mo •

+ Follow ···

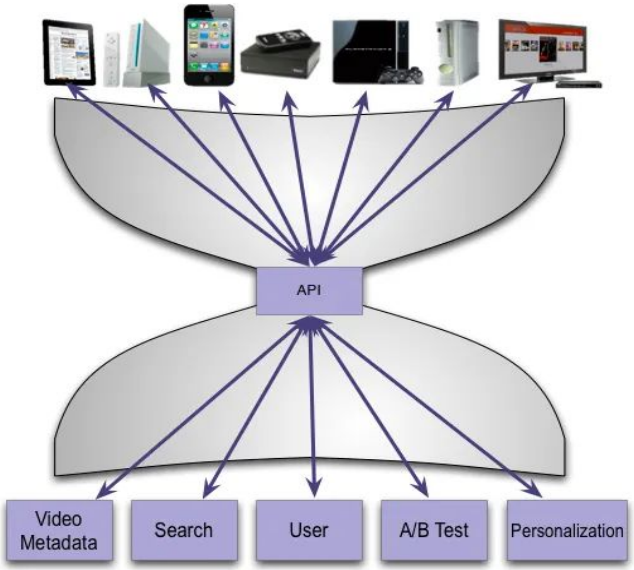
Netflix's Deployment Strategy Unveiled: A Glimpse into Efficiency 🚀

In a realm where weekly deployments are the norm, Netflix shines with a staggering 4,000+ daily deployments, seamlessly executed without a hint of downtime.

The key? Complete automation of tasks unsuitable for human intervention.

Case Study: Netflix (Problem)

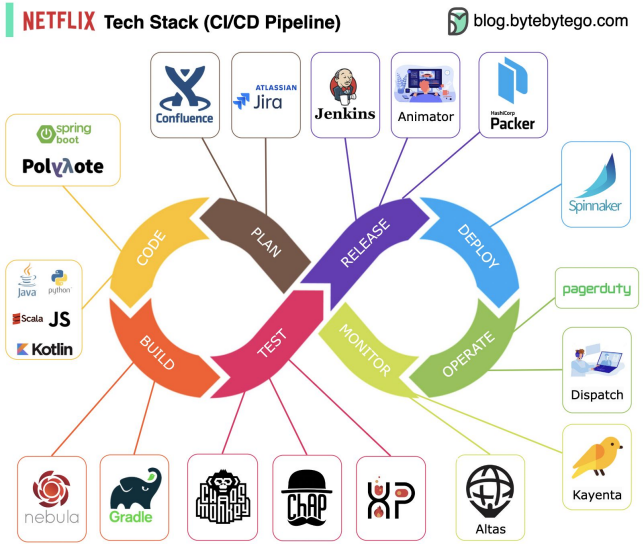
The Challenge:



[Netflix Technology Blog]

Case Study: Netflix (CI/CD)

“To meet demand for new features and to make a growing infrastructure easier to manage, we’ve been overhauling our dev, build, test, and deploy pipeline with an eye toward a continuous delivery…Any process requiring people to execute manual steps repetitively will get you into trouble…Once code has been checked in, the CI servers take over.”



[Netflix Technology Blog]

Case Study: Netflix (Deployment)

Canary Deployment:

- A **canary machine** is launched first with the new software load running real traffic to sanity test the load in a production environment. If the canary doesn't die they move on with the complete upgrade.
- Let the new **cluster bake** for a while.
- If there aren't any problem **tear down the old** cluster and the new cluster is now the operational cluster of record.
- If there are problems **redirect requests to the old** cluster, tear down the new cluster, and figure out what went wrong.
- Downstream services see the same traffic volume, so the process is transparent.

The basic idea of a canary is that you run new code on a small subset of your production infrastructure, for example, 1% of prod traffic, and you see how the new code (the canary) compares to the old code (the baseline)... Failing on a few canary machines is far superior to having a systemic failure across an entire fleet of servers."

[Netflix Technology Blog]

Case Study: Netflix (SE Process)

How does this approach impact the software engineering processes at Netflix?

There's virtually no process at Netflix. They don't believe in it. They don't like to enforce anything. It slows progress and stunts innovation. They want **high velocity** development. Each team can do what they want and release whenever they want, how often they want. Teams release software all the time, independent of each other. They call this an "optimistic" approach to development.

“ ‘It’s Engineering... but not as we know it’… Software Engineering - solution to the software crisis, or part of the problem?” [Bryant]

Class Activity

With your Project 2 team, complete the following activity:

1. **Reflection on CI/CD and development bots**
2. **Quickstart Guide for GitHub Actions (required for PM2.3)**
3. **Mattermost Webhook**
4. **Post-reflection**
5. **Mattermost Bot Project Integration (optional)**

Submit: Attendance will be checked based on summary of reflections (Canvas) *and* ***individual*** **`bot`** post on Mattermost 🤖

References

- RS Pressman. *“Software engineering: a practitioner's approach”.*
- Smith et al. *“State of DevOps 2021”.* Google Cloud
- Accelerate State of DevOps 2024. *“DORA: State of AI-assisted Software Development”*. Google Cloud, https://dora.dev/dora-report-2025
- Harness. *“Intro to Deployment Strategies: Blue-Green, Canary, and More”*.
- Michael Hilton, et al. *“Continuous Integration (CI) Needs and Wishes for Developers of Proprietary Code”.* 2016
- Michael Hilton, et al. *“Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects”.* 2016
- Rohith Lyadalia. “*Why Did Netflix Not Go Down During the AWS Outage?*”. Medium, 2025.
- High Scalability. “*Netflix: Developing, Deploying, and Supporting Software According to the Way of the Cloud*”. 2011
- Taher A. Ghaleb and Dulina Rathnayake. “*Can LLMs Write CI? A Study on Automatic Generation of GitHub Actions Configurations*”. Arxiv 2507.17165, 2025.
- Steve Fenton. *“AI Won’t Fix Your Software Delivery Problems”*. CoderLegion, 2025.
- Dare Obasanjo. *“Dark Launches, Gradual Ramps and Isolation: Testing the Scalability of New Features on your Web Site”.* 2008
- Marko Anastasov. “*Continuous Integration (CI) Explained*”. Semaphore CI, 2025
- Netflix Technology Blog. “*Deploying the Netflix API*”. 2013.
- Netflix Technology Blog. “*Embracing the Differences : Inside the Netflix API Redesign*”. 2012.
- David Gray Widder, et al. *“I’m Leaving You, Travis: A Continuous Integration Breakup Story”*. 2018
- Dharmveer Kumar Yadav, Sandip Dutta & Chandrashekhar Azad. “*Study and Analysis of Test Case Prioritization Technique*”. Lecture Notes in Electrical Engineering, vol 692. Springer, Singapore. 2021.
- Chris Parnin